555F

AVIS IMPORTANT AUX ETUDIANTS

1. Chacune des feuilles de votre copie doit comporter une étiquette code à barres placée à l'endroit indiqué «coller ici votre code à barres».
2. Une copie d'examen comporte une seule «feuille principale» et des «feuilles suites».Sur chacune de vos feuilles,le code à barres est obligatoire.
3. Cette feuille d'examen est strictement personnelle. Elle ne doit comporter aucun signe distinctif. Elle doit être écrite en noir et/ou bleu.
4. Le non respect de l'une de ces recommandations peut faire attribuer la note ZERO à l'épreuve.

Coller ici votre code à barre

NOTE

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20

00	25	50	75

Institut Supérieur d'Informatique et de Mathématiques de Monastir

ISIMM

Devoir Surveillé – S1 – 2026/2027

Filière : *3ème LI*	**Matière :** Développement d'applications réparties	**Enseignant : Marwa Thabet**
Date : 14/ 11 / 2026		**Documents autorisés : Non**
Durée de l'examen : 1h00	Régime d'évaluation : **Mixte**	**Nombre de pages : 7**

Exercice 1

Le club de sécurité informatique de l'ISIMM souhaite développer une application client/serveur en Java avec les **sockets TCP**. Le client saisit un mot de passe au clavier et l'envoie au serveur, qui écoute sur le **port 2025** de la machine locale. Le serveur répond par la chaîne « Mot de passe fort » si le mot de passe contient au moins 8 caractères, sinon par « Mot de passe faible (n caractères) ».

Remarque: Le code est découpé en **blocs nommés** (S1 à S6 pour le serveur, C1 à C6 pour le client). Dans les questions 3 et 4, vous pouvez désigner un bloc par son nom au lieu de copier son code (exemple : « Bloc S5 inchangé »).

1) Compléter les codes du serveur et du client.

```
/********* Coté serveur *********/
// ═══ Bloc S1 : imports et déclaration de la classe ═══
import java.net.*;
import java.io.*;
public class ServeurMdp {
   public static void main(String[] args) {
      try {
// ═══ Bloc S2 : création du socket d'écoute ═══
         ..... ss = new .....(2025);
// ═══ Bloc S3 : attente et acceptation d'une connexion ═══
         System.out.println("J'attends une connexion");
         .....  s = ss. .....();
// ═══ Bloc S4 : création des flux d'échange ═══
         .....  is = s. .....();
         InputStreamReader isr = new InputStreamReader(is);
         BufferedReader br = new .....(isr);
         OutputStream os = s. .....();
         PrintWriter pw = new PrintWriter(os, .....);
// ═══ Bloc S5 : traitement de la requête ═══
         String mdp = br. .....();
         String rep;
         if (.....  .length() >=.....) rep = "Mot de passe fort";
         .....  rep = "Mot de passe faible (" + mdp.length() + " caractères)";
```

```
        pw. .....(rep);
// ═══ Bloc S6 : fermeture du socket de service ═══
        s.close();
    } catch (Exception e) { e.printStackTrace(); }}}
```

```
                                /********* Coté client *********/
// ═══ Bloc C1 : imports et déclaration de la classe ═══
import java.net.*;
import java.io.*;
import java.util.Scanner;
public class ClientMdp {
  public static void main(String[] args) {
    try {
// ═══ Bloc C2 : demande de connexion au serveur ═══
        Socket s = new .....("....................", .....);

// ═══ Bloc C3 : création des flux d'échange ═══
        InputStream is = s.getInputStream();
        BufferedReader br = new BufferedReader(new .....(is));
        OutputStream os = s.getOutputStream();
        ..... pw = new .....(os, true);

// ═══ Bloc C4 : saisie et envoi du mot de passe ═══
        Scanner clavier = new Scanner(System.in);
        System.out.println("Donner un mot de passe");
        String mdp = clavier.next();

        pw.println(.....);

// ═══ Bloc C5 : réception et affichage de la réponse ═══
        String rep = br. .....();

        System.out.println("Réponse du serveur : " + rep);
// ═══ Bloc C6 : fermeture ═══
        s. .....();

    } catch (Exception e) { e.printStackTrace(); }
  }}
```

2) Quel est le rôle de l'objet ss (bloc S2) et celui de l'objet s (bloc S3) côté serveur ?

.....

.....

3) Ce serveur est-il qualifié de mono-client, clients itératifs ou multi-clients ? Est-ce qu'un deuxième client qui tente de se connecter après le premier réussira ?

.....

.....

4) On souhaite que le serveur traite plusieurs clients l'un après l'autre. Réécrire le corps de la méthode main du serveur itératif en utilisant les noms des blocs.

```
                    /********* Coté serveur *********/
try {
   // Bloc S2 inchangé
   .....
      // Bloc S..... inchangé
      // Bloc S..... inchangé
      // .....
      // .....
   }
} catch (Exception e) { e.printStackTrace(); }
```

5) On souhaite maintenant que le serveur traite plusieurs clients simultanément.
   a) Écrire la classe Service en précisant quels blocs du serveur sont déplacés dans run().

```
import java.net.*;
import java.io.*;
public class Service extends Thread {
   private Socket s;
   private int numClient;
   public Service(Socket socket, int numClient) {
      super();
      this.s = socket;
      this.numClient = numClient;
   }
   @Override
   public void run() {
      try {
         System.out.println("Connexion du client num " + numClient);
```

Coller ici votre
code à barre

SUITE

```
        // Bloc ..... inchangé
        // Bloc ..... inchangé
        // Bloc ..... inchangé

    } catch (Exception e) { e.printStackTrace(); }}}
```

b) Réécrire la méthode main du serveur.

```
public static void main(String[] args) {
   int nbClients = 0;
   try {

      // ..........................................................

      while (true) {

         // ..........................................................

         nbClients++;

         ..........................................................

      }} catch (Exception e) { e.printStackTrace(); }}
```

Exercice 2

Compléter le tableau ci-dessous en inscrivant la lettre correspondante à votre réponse pour chaque question du QCM.

Tableau des réponses									
1	2	3	4	5	6	7	8	9	10

1. Dans l'architecture réseau, les sockets se situent :
   a) au-dessus de la couche d'application.
   b) dans la couche réseau, à la place du protocole IP.
   c) juste au-dessus de la couche transport (protocoles TCP ou UDP).
   d) au niveau de la couche physique.
   e) à l'intérieur du protocole HTTP.
2. Quelle affirmation décrit correctement la communication en mode non connecté (UDP) ?
   a) Pas de connexion préalable ni d'accusé de réception ; l'adresse de destination est requise à chaque envoi.
   b) Une connexion durable est établie entre les processus.
   c) Les données arrivent toujours dans l'ordre d'envoi.

d) Le serveur utilise un socket d'écoute et un socket de service par client.
e) Le service est fiable, mais plus lent que TCP.

3. Quelle instruction permet d'attendre puis de récupérer un datagramme UDP ?
a) dgSck.accept();
b) dgSck.receive(dgpRecu);
c) dgSck.read();
d) dgpRecu.getLength();
e) dgSck.send(dgpRecu);

4. Pour recevoir les datagrammes envoyés à un groupe de machines, un processus doit :
a) créer un DatagramSocket puis appeler accept().
b) utiliser une adresse IP de classe A.
c) créer un ServerSocket sur le port 4321.
d) demander à l'émetteur d'envoyer un datagramme séparé à chaque machine.
e) créer un MulticastSocket et s'abonner à une adresse de classe D avec joinGroup().

5. Pour envoyer un objet Compte avec un ObjectOutputStream, il faut que :
a) la classe Compte implémente l'interface Serializable.
b) la classe Compte implémente l'interface Remote.
c) la classe Compte hérite de la classe Thread.
d) l'objet soit envoyé avec un PrintWriter.
e) l'objet soit d'abord converti en entier avec os.write().

6. Le marshalling est :
a) le traitement des erreurs de communication (pertes, délais, pannes).
b) la localisation de l'adresse d'un module distant.
c) la transformation d'une représentation de données particulière en une représentation plus adéquate à la communication.
d) la création d'un thread par client connecté.
e) la fermeture de la connexion entre le client et le serveur.

7. À partir d'une spécification IDL, le compilateur d'interface :
a) exécute le serveur sur la machine distante.
b) génère les stubs (souches) côté client et les skeletons côté serveur.
c) attribue une adresse IP à chaque module.
d) remplace les sockets TCP par des sockets UDP.
e) supprime le besoin de sérialiser les paramètres.

8. Un serveur itératif sert un client A qui met beaucoup de temps à envoyer son mot de passe. Pendant ce temps, un client B se connecte. Que se passe-t-il ?
a) B est servi immédiatement dans un nouveau thread.
b) B doit attendre : le serveur reste bloqué sur readLine() pour A et ne revient à accept() qu'après avoir terminé avec A.
c) Le serveur interrompt A pour servir B en priorité.
d) La connexion de B provoque l'arrêt du serveur (BindException).
e) B reçoit la réponse destinée à A.

9. Dans un serveur multi-threads, que se passe-t-il à chaque nouvelle connexion acceptée ?
a) Un nouveau ServerSocket est créé sur un nouveau port.
b) Le serveur principal réalise lui-même les lectures et écritures, puis revient à accept().
c) Tous les clients partagent le même socket de service.
d) Le thread principal se termine et le nouveau thread devient le serveur.
e) Un nouveau thread est créé et associé au socket du client ; il gère les échanges avec ce client pendant que le serveur principal continue à accepter d'autres connexions.

10. Quel schéma représente correctement le déroulement de l'aller d'un appel de procédure à distance (RPC), du client jusqu'au serveur ?

Schéma A

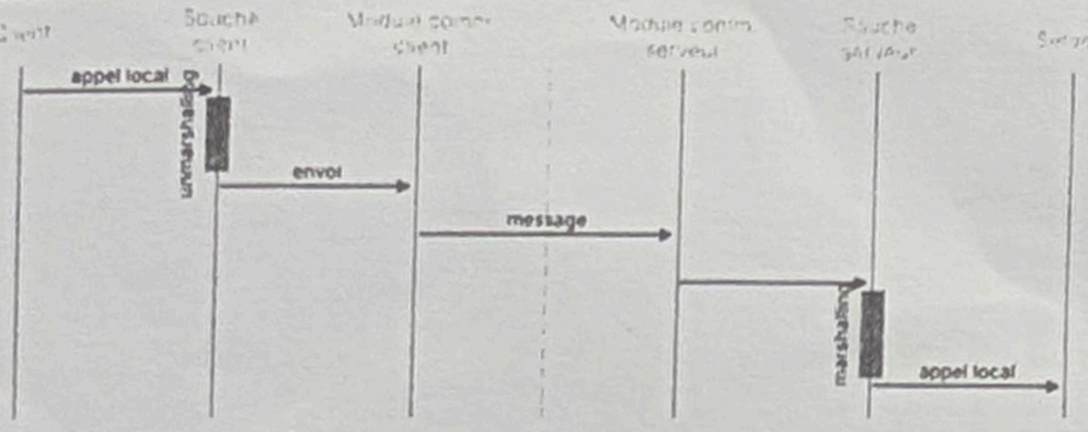


Schéma B

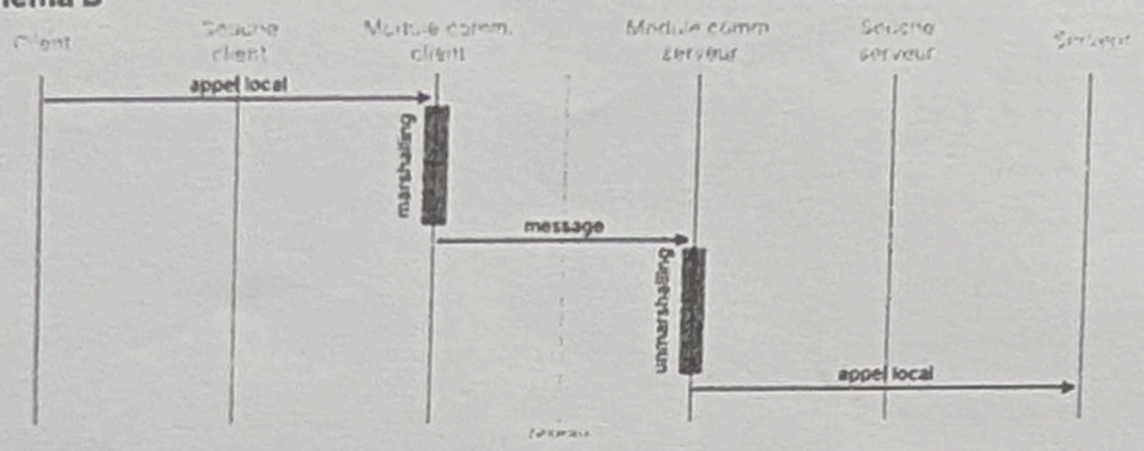


Schéma C

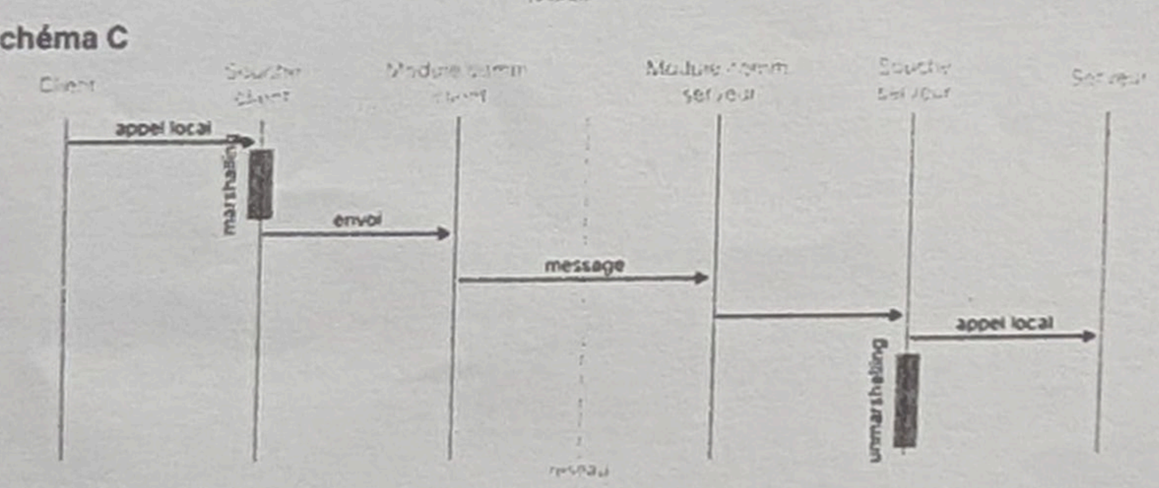


Schéma D

